Basic (cold recall) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried answering out loud or in writing first.

- Hint 1 — The idea, and the exact shape
- Hint 2 — The plan, and almost the whole answer

Hint 1 — The idea, and the exact shape

The skill being tested here isn't knowledge — it's **retrieval under time pressure with no scaffolding**. This is exactly what a real interviewer is checking in the first 10 minutes: not whether you can produce a correct answer given time to think, but whether the fact is actually consolidated in memory, not just recognizable when you see it written down.

This exercise trains specifically the **first** rung of this document's own Four-Depth Format — the 30-second answer. That rung has a fixed shape: (1) a direct one-line claim, (2) the one reason that claim is true, (3) stop — don't pad it with Normal-depth or Deep-depth material that hasn't been asked for yet. Over-answering a cold-recall question is itself a signal to an interviewer that you don't know which parts matter most, the same way under-answering is.

Draft your one-line claim and one-line reason now, for the Doc01 logging-vs-`print()` topic, before Hint 2.

When the follow-up lands early:

Think about what actually happens the instant you finish a good 30-second answer in a real interview: the interviewer usually doesn't nod and move on. They fire a follow-up immediately — "okay, but why not just fake severity levels with `print()` and some `if` statements?" — before you've had a second to breathe, let alone consult notes you don't have.

A cold-recall answer that only exists as a memorized 30-second script falls apart here, because the candidate never built anything past rung 1 of the Four-Depth Format. The real skill this level trains is **pivoting from a finished 30-second answer straight into a Normal-depth answer, on the spot, using the same claim you just gave as the anchor** — not starting over, not stalling, not repeating the claim louder.

The move: treat your one-line "mechanism" from the 30-second answer as a thread to pull, not a closed statement. If your claim was "logging lets you control verbosity without changing code," the instant follow-up thread is *how, specifically* — which pulls straight into levels and handlers, the Normal-depth material, without you needing to have pre-scripted the transition.

The real design question isn't "what's my 30-second answer" — it's "what's the very next sentence I'd say if nobody let me stop, and does it flow from my claim instead of restarting from zero?"

Sketch that next sentence yourself, for the logging topic, before checking Hint 2.

Hint 2 — The plan, and almost the whole answer

The same plan, closer to how you'd actually rehearse it, tied to the Four-Depth Format:

```
select topic: "why logging beats print() in production" (Doc01)

rung 1 (30-second answer):
    claim: one sentence, stated first, no wind-up
    mechanism: one sentence, the specific reason
    stop: no third sentence unless the interviewer asks for more

self-check after answering:
    did the claim come in the first sentence, not buried at the end?
    would a stranger who's never read Doc01 understand the claim on its own?
    did I stay near 30 seconds?
```

Notice "claim first" is doing real work here — a rambling build-up that arrives at the point last is the single most common way a correct answer still reads as weak.

Fill this in for the logging topic yourself, say it out loud once, then compare against the Solution.

Under pushback:

Here's the plan for the curveball case — the interviewer doesn't let your 30-second answer stand, and pushes back immediately:

```
rung 1 (30-second answer): claim + mechanism + stop, as above

interviewer, instantly: "why not just fake it with print() and if statements?"

your move:
    do NOT repeat the claim louder
    do NOT say "let me think" and go silent
    pull the thread from your own mechanism sentence into rung 2 (Normal):
        name the SPECIFIC thing print()-plus-ifs can't do that logging does
```

```
        (e.g. per-module control, output routing to multiple destinations
        at once, no code redeploy needed to change verbosity)
    stop again once that's answered — don't keep climbing to rung 3 or 4
    unless pushed again
```

Try answering the curveball follow-up yourself, using your own claim from Hint 1 as the starting thread, then compare against the Solution.

Full solution: Show me the solution